

SMART LIBRARY MANAGEMENT SYSTEM

A Java-Based Object-Oriented Library Management Application

Course	Java
Project Type	Build Your Own Project
Technology	Java, OOP, Collections, JUnit 5
Academic Year	Fall Semester 2026-27
Student Name	Shaleen Dutta
Registration Number	25BAI10034

Submitted as part of the VITyarthi Build Your Own Project evaluation.

1. Introduction

The Smart Library Management System is a console-based Java application designed to demonstrate objectoriented programming through a practical library scenario. It organizes books, users, issue/return operations, searching, transaction history, and overdue fine calculation.

The implementation is modular, separating model classes from service classes and custom exceptions. It demonstrates encapsulation, inheritance, abstraction, collections, exception handling, and the Java Time API.

2. Problem Statement

Manual library management can make it difficult to track book availability, user records, issue/return operations, transaction history, and overdue fines. The proposed system provides a structured digital approach using Java through a simple menu-driven interface.

3. Objectives

- Develop a practical Java application using object-oriented programming.
- Demonstrate encapsulation, inheritance, abstraction, and polymorphism.
- Provide multiple functional modules with a clear user workflow.
- Implement validation and custom exception handling.
- Use modular packages and separate service classes.
- Provide automated tests for important operations.

4. Functional Requirements

ID	Module	Input	Output
FR-01	Book Management	Book ID, title, author, category	Book record/status
FR-02	User Management	User details	User record
FR-03	Book Search	Keyword	Matching books
FR-04	Issue Book	Book ID, User ID	Confirmation/error
FR-05	Return Book	Book ID	Return confirmation/fine
FR-06	Transactions	None	Transaction history
FR-07	Validation	User input	Validated result/error

4.1 Non-Functional Requirements

Requirement	Description
Usability	Clear menu-driven interface.
Reliability	Exceptions prevent invalid operations from crashing the program.
Maintainability	Separate model, service, exception and utility packages.
Performance	In-memory collections suit the academic-scale dataset.
Resource Efficiency	Lightweight console application without a database server.
Error Handling	Invalid IDs, unavailable books and invalid input are reported clearly.

5. System Architecture

The application uses a simple layered structure: Main.java provides the console interface, service classes contain application operations, model classes represent domain objects, and exception/utility packages support controlled error handling.

Architecture Diagram

USER ■ ▼ Main.java — Console / Menu Layer ■ ▼ Service Layer ■■■ LibraryService ■■■ BookService ■■■ UserService ■ ▼ Model Layer ■■■ Book ■■■ User ■■■ Student / Librarian ■■■ Transaction ■ ■■■ Exception Layer ■ ■■■ BookNotFoundException ■ ■■■ BookUnavailableException ■ ■■■ Utility Layer ■■■ InputValidator

5.1 Process Workflow

Workflow Diagram

START → Display Menu → Select Operation → Read & Validate Input ↓ Invalid? ■■YES■■■ Display Error ■■■ Main Menu ■ NO ▼ Call Service Method → Process Book/User/Transaction ↓ Exception? ■■YES■■■ Display Error ■■■ Main Menu ■ NO ▼ Display Result → Main Menu → Exit → END

6. UML and Design Diagrams

6.1 Use Case Diagram

Use Case Diagram

6.2 Class Diagram

```

    ■■■■■■■■■■■■■■■■■■■■■■ <> User ■■■■■■■■■■■■■■■■■■■■■■ ■ userId, name,email ■
    ■ + getRole() ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■ ▼▼ Student Librarian Book
    ■■■■■■■■■■■■■■■■■■■■■
Transaction(bookId,title,(transactionId,author,category,bookId,userId,available)issueDate,returnDate)LibraryService ■■■
addBook / addUser ■■■ search ■■■ issueBook ■■■ returnBook

```

```

graph LR
    User --> Main: Select Issue Book
    Main --> LibraryService: issueBook(bookId,userId)
    LibraryService --> UserList: Validate user
    UserList --> LibraryService: Find/check availability
    LibraryService --> Book: Available
    Book --> Transaction: Create transaction
    Transaction --> LibraryService: Set unavailable
    LibraryService --> Main: Success
    Main --> User: Display confirmation

```

USER(User_ID PK, Name, Email, Role, Course) BOOK(Book_ID PK, Title, Author, Category, Available)
 TRANSACTION(Transaction_ID PK, Book_ID FK, User_ID FK, Issue_Date, Return_Date) USER 1 ■■■■■■<
 TRANSACTION >■■■■■■■ 1 BOOK Current implementation: in-memory Java collections; the above is the logical
 model for future persistent storage.

Decision	Rationale
Encapsulation	Private fields with controlled accessors.
Inheritance	Student and Librarian extend User.
Abstraction	User represents shared user behavior.
Collections	ArrayList is suitable for the small academic dataset.
Decision	Rationale
Exceptions	Custom exceptions make invalid operations explicit.
Separation of Concerns	Models, services, exceptions and utilities are separated.
Date API	LocalDate/ChronoUnit support date and time calculations.

The prepared project contains Java source files for models, services, custom exceptions, utilities and the main application, plus JUnit testing and GitHub documentation. The Java source was compilation-checked before packaging.

```
SmartLibraryManagementSystem/ ■■■ src/Main.java ■■■
src/model/{Book,User,Student,Librarian,Transaction}.java ■■■
src/service/{LibraryService,BookService,UserService}.java ■■■
src/exception/{BookNotFoundException,BookUnavailableException}.java ■■■ src/util/InputValidator.java
```

```
■■■■ test/LibraryServiceTest.java ■■■■ docs/PROJECT_REPORT.md ■■■■ README.md ■■■■ statement.md ■■■■  
pom.xml ■■■■ .gitignore
```

8.2 Core Operations

- **Issue:** validates the user, locates the book, checks availability, marks it unavailable, and creates a transaction.
- **Return:** locates the active transaction, records the return date, makes the book available, and calculates any fine.
- **Search:** performs case-insensitive matching against title, author and category.
- **Fine:** permits 14 days and charges Rs. 5 per day beyond that period.
- **Error handling:** custom exceptions and safe input parsing keep the application running.

9. Testing Approach

ID	Scenario	Expected Result
T01	Issue available book	Book unavailable; transaction created
T02	Issue same book twice	BookUnavailableException
T03	Return issued book	Book available; return date recorded
T04	Search by keyword	Matching books displayed
T05	Invalid menu input	Validation message; program continues
T06	Unknown book ID	BookNotFoundException

Verification: Java source compilation was successfully checked during project preparation.

10. Screenshots / Results

```
Enter choice: 1
ID      Title                Author          Category        Status
101     Clean Code                Robert C. Martin Programming      Available
102     Java: The Complete Reference Herbert Schildt   Java            Available
103     Data Structures           Seymour Lipschutz Computer Science Available
104     Effective Java             Joshua Bloch     Java            Available
65      Java Programming          Herbert Schildt   Programming      Available

===== SMART LIBRARY MANAGEMENT SYSTEM =====
1. View all books
2. Search books
3. Issue a book
4. Return a book
5. View users
6. View transactions
7. Add a book
8. Exit
```

```
Enter choice: 2
Enter title/author/category: Joshua Bloch
104     Effective Java             Joshua Bloch     Java            Available
```

```
Enter choice: 3
Enter book ID: 101
Enter user ID: 1
Book issued successfully.
```

```
Enter choice: 4
Enter book ID: 101
Book returned successfully. Fine: Rs.0
```

```
Enter choice: 5

ID | Name | Email | Role | Course
1 | Aarav | aarav@example.com | Student | B.Tech CSE
2 | Riya | riya@example.com | Student | B.Tech IT
3 | Library Admin | admin@example.com | Librarian
```

```
Enter choice: 6
Transaction 1 | Book: 101 | User: 1 | Issued: 2026-09-18 | Returned: 2026-09-18 | Fine: Rs.0
Transaction 2 | Book: 102 | User: 2 | Issued: 2026-09-18 | Returned: Not returned | Fine: Rs.0
Transaction 3 | Book: 65 | User: 3 | Issued: 2026-09-18 | Returned: Not returned | Fine: Rs.0
```

```
Enter choice: 7
Book ID: 107
Title: Introduction to problem solving technique
Author: Rasputin
Category: Programming
Book added successfully.
```

```
Enter choice: 8
Thank you for using Smart Library Management System.
```

11. Challenges Faced

- Designing relationships among books, users and transactions.
- Handling unavailable books and invalid returns without terminating the application.
- Keeping the implementation modular while demonstrating Java OOP concepts.
- Implementing date-based fine calculation.
- Writing service methods that can be tested independently.

12. Learnings and Key Takeaways

- Applied encapsulation, inheritance, abstraction and polymorphism.
- Used Java Collections Framework for in-memory data management.

- Created and used custom checked exceptions.
- Used LocalDate and ChronoUnit for date calculations.
- Organized code into packages and service classes.
- Applied unit testing to verify service behavior.



13. Future Enhancements

- Add MySQL or SQLite persistence.
- Add login and role-based access control.
- Develop a JavaFX graphical interface.
- Persist transaction history across restarts.
- Add due-date reminders and notifications.
- Introduce advanced reports and dashboards.
- Add book reservation and wait-list functionality.

14. References

- Java SE API documentation and standard Java library concepts used in the implementation.
- Course lectures, notes and laboratory material for Java and object-oriented programming.
- VITYarathi – Build Your Own Project: General Project Instructions & Submission Guidelines (provided coursedocument).

Appendix — Viva / Demonstration Points

- Explain why User is abstract and why Student/Librarian inherit from it.
- Explain how LibraryService separates business logic from Main.java.
- Demonstrate what happens when an unavailable book is issued.
- Explain the fine calculation.
- Show the JUnit tests and what each verifies.
- Explain functional versus non-functional requirements.
- Explain the relationship among User, Book and Transaction.